

NutriLabelAI - Complete Repository Guide

Project Name: NutriLabelAI

Repository: github.com/yvagula06/Senior-Design-2025

Description: AI-powered nutrition estimation system with semantic search, camera-based meal detection, mobile app, and comprehensive food database

Project Overview

NutriLabelAI is a production-ready nutrition estimation system that helps users get accurate nutrition information for any dish by combining semantic search with a comprehensive food database.

What It Does

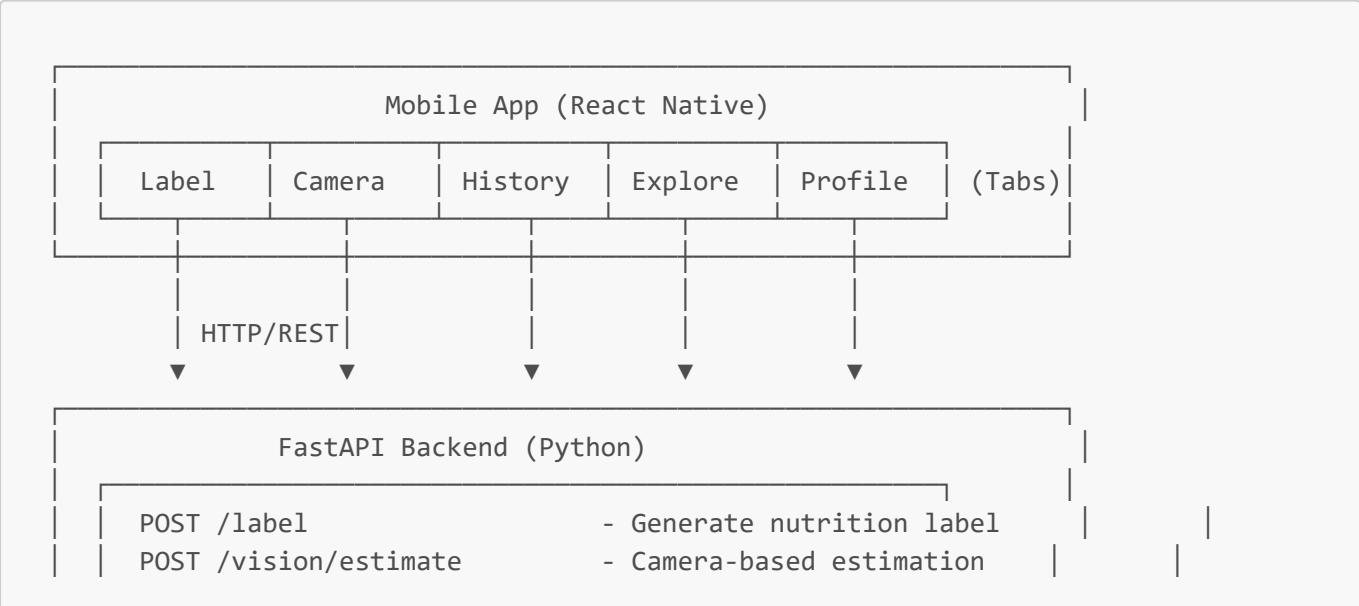
Text-Based Label Generation: Enter a dish name like "chicken tikka masala" or "Big Mac" and get:

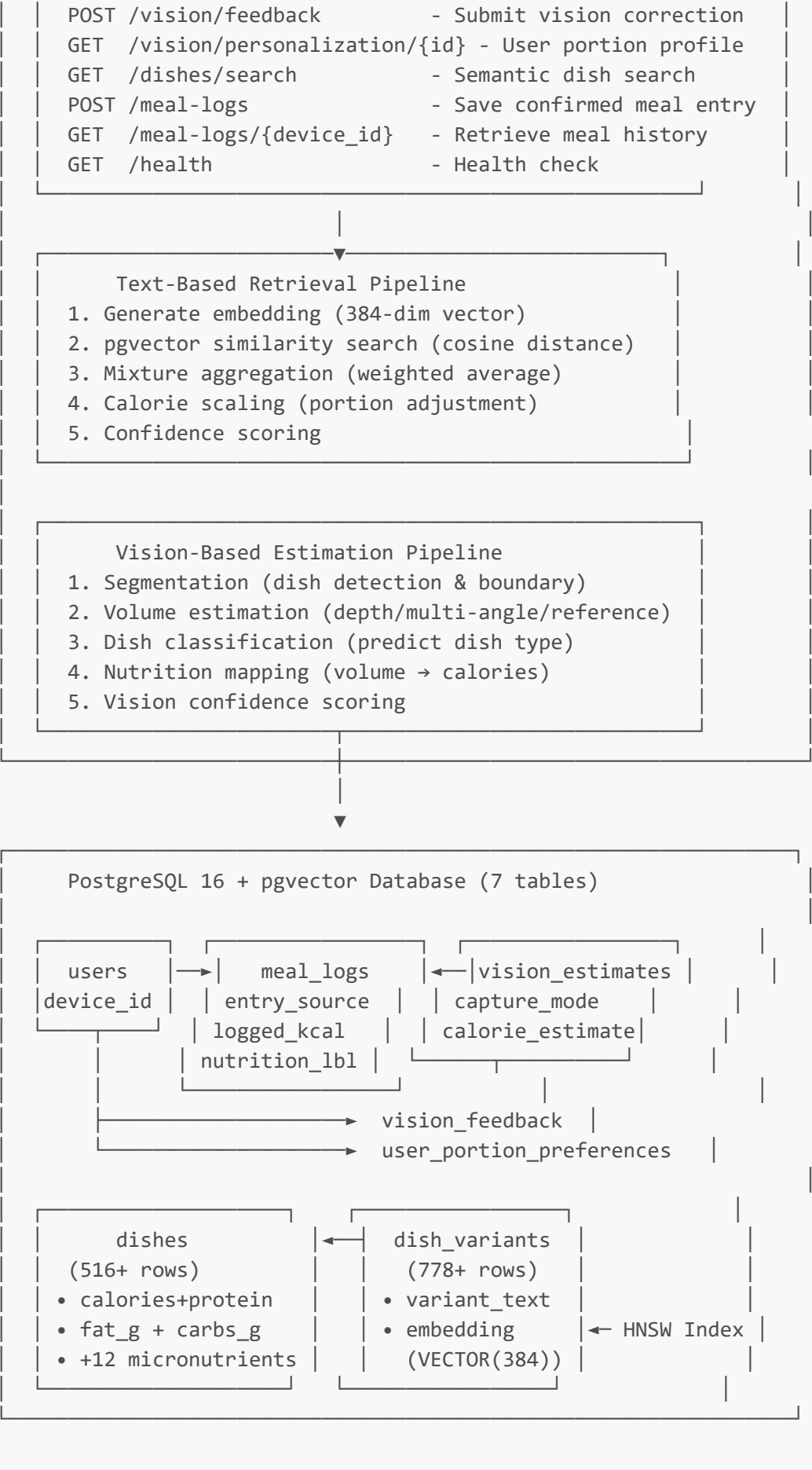
- ✓ Full FDA-style nutrition label: **calories, protein_g, carbs_g, fat_g** (required) plus up to 12 nullable micronutrients: **fiber_g, sugar_g, sodium_mg, potassium_mg, saturated_fat_g, trans_fat_g, cholesterol_mg, vitamin_a_mcg, vitamin_c_mg, vitamin_d_mcg, calcium_mg, iron_mg**
- ✓ Confidence score indicating reliability
- ✓ Best match from 516+ dishes in database
- ✓ Instant results via mobile app or API
- ✓ Optional meal log creation: when `device_id` is provided the response includes a `meal_log_id`

Camera-Based Meal Estimation: Take a photo of your meal and get:

- ✓ Automatic dish identification from image
- ✓ Volume/portion size estimation
- ✓ Calorie estimates with accuracy ranges
- ✓ Multiple estimation modes (depth, multi-angle, reference-based)

System Architecture





Core Features

- 🍽️ **516+ dishes** covering fast food, restaurants, home cooking, and international cuisines

- 🔍 **Semantic search** - finds dishes even with typos, synonyms, or different phrasings
- 📷 **Camera-based estimation** - take a photo to get nutrition estimates with volume detection
- 📊 **Complete nutrition** - up to 16 FDA nutrients per dish (4 required macros + up to 12 nullable micronutrients: fiber, sugar, sodium, potassium, saturated fat, trans fat, cholesterol, vitamins A/C/D, calcium, iron)
- 🎯 **Confidence scoring** - tells you how reliable the match is (0.0-1.0)
- 📱 **Native mobile app** - iOS & Android via React Native + Expo
- ⚡ **Fast retrieval** - sub-100ms queries with pgvector HNSW indexing
- 📏 **Portion scaling** - adjust nutrition based on target calories

🏗️ System Architecture

Three-Tier Architecture

1. Presentation Layer - Mobile App

- React Native + Expo for cross-platform support
- TypeScript for type safety
- React Native Paper for Material Design UI
- Expo Go for instant testing on physical devices

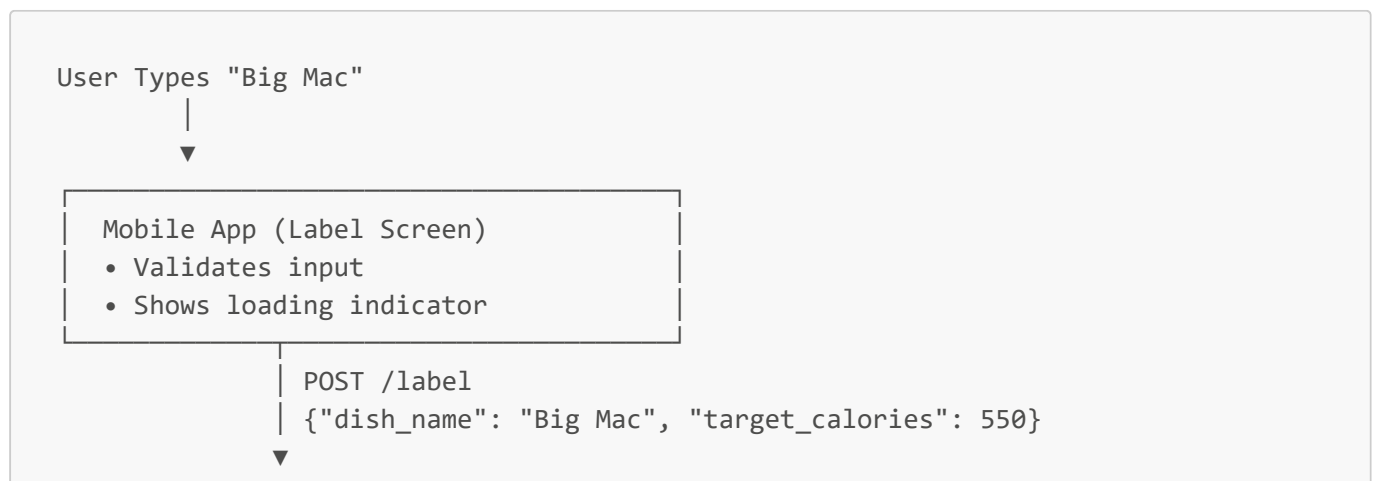
2. Application Layer - FastAPI Backend

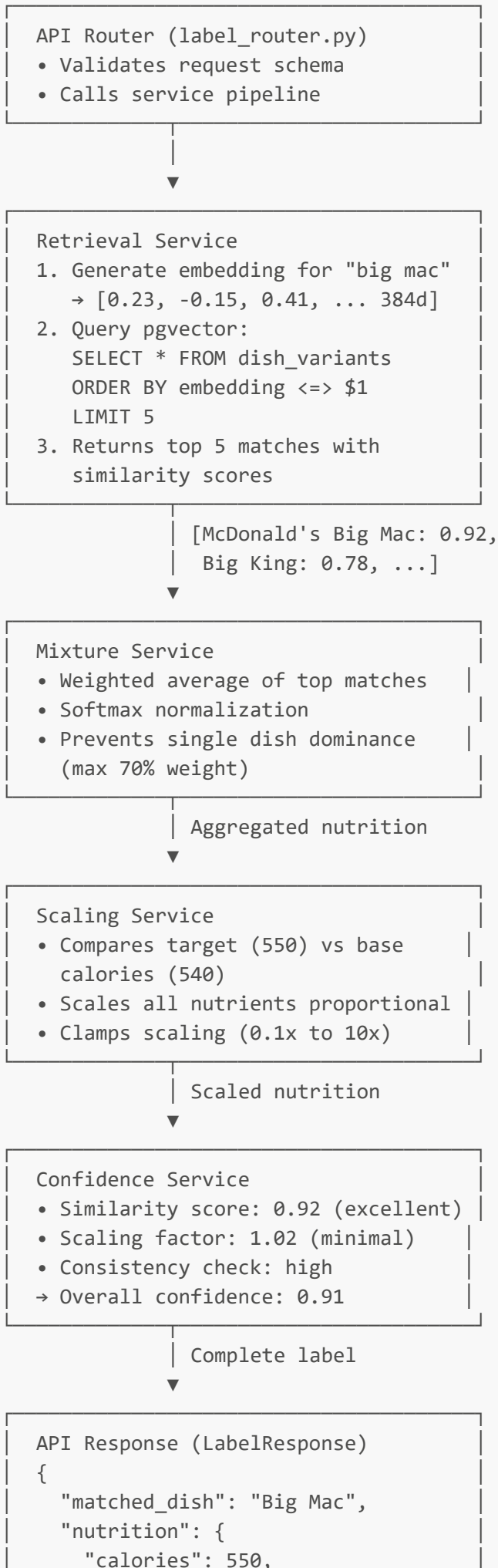
- RESTful API with automatic OpenAPI documentation
- Service-oriented architecture (retrieval, scaling, confidence services)
- Sentence-transformers for embedding generation
- Pydantic for request/response validation

3. Data Layer - PostgreSQL + pgvector

- Relational database with vector extension — **7 canonical tables**: `users`, `dishes`, `dish_variants`, `meal_logs`, `vision_estimates`, `vision_feedback`, `user_portion_preferences`
- HNSW cosine index on `dish_variants.embedding` for sub-100ms semantic search
- Alembic for version-controlled schema migrations
- Foreign key relationships for data integrity (`SET NULL` on cross-table FKs to preserve history)

How It Works: Label Generation Flow





```
"protein_g": 25.5,
"carbs_g": 45.2,
"fat_g": 29.0,
"fiber_g": 3.0,
"sugar_g": 9.0,
"sodium_mg": 1010,
"potassium_mg": null,
"saturated_fat_g": 10.0,
"trans_fat_g": 1.0,
"cholesterol_mg": 80,
"vitamin_a_mcg": null,
"vitamin_c_mg": null,
"vitamin_d_mcg": null,
"calcium_mg": null,
"iron_mg": null
},
"confidence": 0.91,
"explanation": "Excellent match",
"meal_log_id": 101
}
NOTE: meal_log_id is non-null only
when device_id was in the request.
```

JSON response

- Mobile App (Result Screen)
- Displays full nutrition label
 - Shows confidence badge
 - If device_id was sent, meal was auto-saved to meal_logs table.
 - Client may also call POST /meal-logs explicitly to save.

Complete Directory Structure

```
Senior-Design-2025/
├── mobile/ # React Native Mobile App
│   ├── src/
│   │   ├── components/ # Reusable UI components
│   │   │   ├── Button.tsx # Custom buttons
│   │   │   ├── Card.tsx # Content cards
│   │   │   ├── TextInput.tsx # Text inputs
│   │   │   ├── LoadingSpinner.tsx # Loading indicator
│   │   │   ├── MacroPieChart.tsx # Macro distribution chart
│   │   │   ├── index.ts # Component exports
│   │   │   └── Explore/ # Explore-specific components
│   │   │       └── CategoryHeader.tsx
```

```

├── DishCard.tsx
├── index.ts
├── History/ # History-specific components
│   ├── HistoryItemCard.tsx
│   └── index.ts
├── Label/ # Label-specific components
│   ├── ConfidenceBar.tsx
│   ├── DishSearchInput.tsx
│   ├── NutritionLabelCard.tsx
│   ├── VariantBottomSheet.tsx
│   ├── VariantDrawerButton.tsx
│   └── index.ts
├── Toast.tsx # Toast notification component
├── Profile/ # Profile-specific components
│   ├── InfoCard.tsx
│   ├── SectionHeader.tsx
│   ├── SettingsItem.tsx
│   └── index.ts
├── Vision/ # Vision-specific components
│   ├── AngleIndicator.tsx
│   ├── ARScanningOverlay.tsx
│   ├── CalorieRangeDisplay.tsx
│   ├── CameraGuide.tsx
│   ├── DishPredictionList.tsx
│   └── index.ts
├── config/ # App configuration
│   └── fonts.ts # Font definitions
├── context/ # State management
│   ├── FoodContext.tsx # Global food state
│   └── ThemeContext.tsx # Theme/dark-mode state
├── hooks/ # Custom React hooks
│   ├── index.ts # Hook exports
│   └── useDepthCamera.ts # Depth camera hook
├── native/ # Native module bridges
│   └── DepthExtractor.ts # Native depth data extraction
├── navigation/ # Navigation setup
│   ├── RootTabNavigator.tsx # Root tab navigator
│   ├── BottomTabNavigator.tsx # Bottom tab bar
│   ├── LabelStackNavigator.tsx # Label flow stack
│   ├── ExploreStackNavigator.tsx # Explore flow stack
│   ├── HistoryStackNavigator.tsx # History flow stack
│   └── types.ts # Navigation type definitions
├── screens/ # Main app screens
│   ├── Label/ # Label generation screens
│   │   ├── LabelHomeScreen.tsx # Input form
│   │   ├── LabelResultScreen.tsx # Results display
│   │   └── BarcodeScannerScreen.tsx # Barcode scan interface
│   └── Vision/ # Camera-based screens

```

```

├── CameraCaptureScreen.tsx # Camera interface
├── EstimationResultScreen.tsx # Vision results
├── History/                # History screens
│   ├── HistoryListScreen.tsx # List view
│   └── HistoryDetailScreen.tsx # Detail view
├── AddEntryScreen.tsx      # Manual food entry
├── DailyConsumerScreen.tsx # Daily tracking/history
├── ExploreScreen.tsx       # Browse dishes
├── ProfileScreen.tsx       # User profile
├── index.ts                # Screen exports
├── services/               # API communication
│   ├── api.ts              # Axios client config
│   ├── barcode.ts          # Barcode scanning service
│   ├── label.ts            # Label API calls
│   ├── labelApi.ts         # Label service
│   ├── visionApi.ts        # Vision API calls
│   ├── DepthExtractor.ts   # Depth data extraction service
│   └── storage.ts          # AsyncStorage wrapper
├── theme/                  # Design system
│   ├── animations.ts       # Animation definitions
│   ├── colors.ts           # Color palette
│   ├── constants.ts        # Theme constants
│   ├── typography.ts       # Typography styles
│   └── index.ts            # Theme exports
├── types/                  # TypeScript definitions
│   ├── label.ts            # Label types
│   ├── nutrition.ts        # Nutrition data types
│   └── vision.ts           # Vision/camera types
├── assets/                 # Static assets
│   ├── fonts/              # Custom fonts
│   └── images/              # Images & icons
├── App.tsx                 # Root component
├── app.json                # Expo config
├── package.json            # Dependencies
├── tsconfig.json           # TypeScript config
├── README.md               # Mobile documentation
├── 🔧 app/                  # FastAPI Backend
│   ├── api/                # API route handlers
│   │   ├── __init__.py
│   │   └── dishes_router.py # GET /dishes/search?q=&k= - Semantic dish
├── search
│   ├── label_router.py     # POST /label - Generate labels ★
│   └── vision_router.py    # POST /vision/estimate - Camera estimation
│                           # POST /vision/feedback - Submit correction
│                           # GET /vision/personalization/{id} - User
├── profile
├── stats
│   └── GET /vision/feedback/stats - Aggregated

```

```

└─ feedback_router.py      # POST /meal-logs - Save meal log ★
                           # GET  /meal-logs/{device_id} - Retrieve

history
└─ core/                  # Configuration
    └─ __init__.py
    └─ settings.py        # Environment variables

└─ db/                    # Database layer
    └─ __init__.py
    └─ models.py          # SQLAlchemy ORM models ★ (7 tables)
                           # - User, Dish, DishVariant
                           # - MealLog, VisionEstimate
                           # - VisionFeedback, UserPortionPreference
    └─ session.py         # Database connection

└─ schemas/              # Pydantic schemas
    └─ __init__.py
    └─ label.py           # Request/response models ★
                           # - LabelRequest
                           # - LabelResponse
    └─ vision.py          # Vision request/response models
                           # - VisionRequest
                           # - VisionResponse

└─ services/              # Business logic ★
    └─ __init__.py
    └─ retrieval_service.py # pgvector similarity search
    └─ mixture_service.py  # Weighted aggregation
    └─ scaling_service.py  # Calorie-based scaling
    └─ rebalance_service.py # Nutrient rebalancing
    └─ confidence_service.py # Confidence scoring
    └─ vision_orchestrator.py # Vision pipeline orchestration
    └─ vision_api_client.py # External vision API client
    └─ vision_feedback_service.py # Vision feedback tracking
    └─ segmentation_service.py # Dish segmentation from images
    └─ volume_estimator.py   # Volume/portion size estimation
    └─ depth_volume_estimator.py # Depth-based volume estimation
    └─ dish_classifier.py    # Dish type classification
    └─ nutrition_mapper.py   # Volume to nutrition mapping
    └─ vision_confidence_adapter.py # Vision confidence scoring

└─ utils/                 # Utilities
    └─ __init__.py
    └─ embeddings.py       # Sentence-transformer loader

└─ tests/                 # Unit tests
    └─ test_confidence.py  # Confidence service tests
    └─ test_health.py      # Health endpoint tests
    └─ test_label_flow.py  # End-to-end tests
    └─ test_rebalance.py   # Rebalance tests
    └─ test_confidence_bounds.py # Boundary tests
    └─ test_end_to_end.py  # Integration tests
    └─ test_label_router.py # Router tests

```

- └─ test_scaling_edge_cases.py # Edge case tests
- └─ test_vision_estimate_contract.py # Vision API contract tests
- └─ test_vision_orchestrator_mocked.py # Vision orchestrator unit tests
- └─ __init__.py
- └─ main.py # FastAPI app entry point ★
- ─ alembic/ # Database Migrations
 - └─ versions/
 - └─ 0001_init_schema_with_pgvector.py ★
 - # Creates ALL 7 tables: users, dishes, dish_variants,
 - # meal_logs, vision_estimates, vision_feedback,
 - # user_portion_preferences
 - # Installs pgvector extension + HNSW cosine index
 - # Drops any legacy tables before creating canonical schema
 - # Installs _update_updated_at trigger (users/dishes/variants)
 - └─ 0002_add_vision_feedback_schema.py
 - # No-op placeholder (revision: 0002_noop)
 - # All tables live in 0001_canonical; keeps revision chain intact
 - └─ env.py # Migration environment
- ─ data/ # Datasets ★
 - └─ seed_dishes.csv # Curated dishes (1 row)
 - └─ fastfood.csv # Fast food items (515 rows)
 - └─ usda_branded_foods.csv # USDA database (full)
 - └─ usda_branded_foods_reduced.csv # USDA subset
- ─ Implementation_Plans/ # Feature design documents
 - └─ Camera_Functionality_Plan.md # Camera pipeline architecture plan
- ─ Poster/ # Academic poster assets
 - └─ diagrams.py # Diagram generation script
 - └─ diagrams_copy.py # Diagram variant script
 - └─ architecture_diagram.png # Architecture diagram (PNG)
 - └─ architecture_diagram.svg # Architecture diagram (SVG)
 - └─ custom_pipeline.svg # Custom pipeline graphic (SVG)
 - └─ pipeline_flow_diagram.png # Pipeline flow (PNG)
 - └─ pipeline_flow_diagram.svg # Pipeline flow (SVG)
 - └─ pipeline_vertical_zigzag.png # Vertical pipeline layout (PNG)
 - └─ pipeline_vertical_zigzag.svg # Vertical pipeline layout (SVG)
- ─ ml_models/ # Trained Models
 - └─ neural_network_model.pth # PyTorch MLP (archived)
- ─ scripts/ # Data & Admin Scripts ★
 - └─ ingest_seed.py # Load seed dishes
 - └─ embed_dishes.py # Generate embeddings
 - └─ import_usda_dishes.py # Populate dishes from USDA data ★★
 - └─ import_usda_fixed.py # Fixed USDA import variant
 - └─ import_usda_whole_foods.py # Import USDA whole foods subset
 - └─ import_fastfood.py # Import fast food dataset
 - └─ import_openfoodfacts.py # Import Open Food Facts data
 - └─ import_continuous.ps1 # Continuous import PowerShell script
 - └─ populate_db_simple.py # Simplified database population

└─	reduce_dataset.py	# Preprocess CSVs
└─	inspect_db.py	# Database inspector
└─	query_db.py	# Test queries
└─	check_dishes.py	# Validate data
└─	insert_test_data.py	# Test records
└─	inspect_audit_logs.py	# Audit logs
└─	README_PREPROCESSING.md	# Preprocessing guide
└─	 Jupyter Notebooks	# ML Experimentation
└─	DSA330_Nutrition_TextRegression.ipynb	# Text regression experiments
└─	 Visualization Assets	
└─	pipeline_horizontal.png	# System pipeline diagram (PNG)
└─	pipeline_horizontal.svg	# System pipeline diagram (SVG)
└─	system_architecture.png	# Architecture overview (PNG)
└─	system_architecture.svg	# Architecture overview (SVG)
└─	 Configuration Files	
└─	docker-compose.yml	# Docker orchestration ★
		# - nutrition_db (PostgreSQL)
		# - nutrition_api (FastAPI)
└─	Dockerfile	# Backend container image
└─	.dockerignore	# Docker ignore patterns
└─	pyproject.toml	# Python dependencies
└─	package.json	# Node.js dependencies
└─	tsconfig.json	# TypeScript config
└─	alembic.ini	# Alembic config
└─	Makefile	# Build automation
└─	.gitignore	# Git ignore patterns
└─	 Documentation ★	
└─	README.md	# Main project overview
└─	QUICKSTART.md	# Complete setup walkthrough ★★
└─	QUICK_START_PHASE2.md	# Phase 2 quick start
└─	REPO_BREAKDOWN.md	# This file ★★
└─	REPO_BREAKDOWN.pdf	# PDF export of this file
└─	DATASET_PLAN.md	# Data acquisition strategy
└─	MOBILE_INTEGRATION.md	# Mobile-backend integration
└─	LABEL_ROUTER_API.md	# API documentation
└─	TROUBLESHOOTING.md	# Common issues & fixes
└─	CAMERA_SETUP_GUIDE.md	# Camera feature setup
└─	CAMERA_STATUS_AND_TODO.md	# Camera implementation status
└─	ANDROID_DEPTH_INTEGRATION.md	# Android depth sensor integration
└─	IOS_SETUP_PHASE2.md	# iOS phase 2 setup
└─	PHASE2_SETUP_GUIDE.md	# Phase 2 setup instructions
└─	PHASE2_IMPLEMENTATION_STEPS.md	# Phase 2 implementation plan
└─	PHASE2_STATUS.md	# Phase 2 completion status
└─	PHASE3_SETUP_GUIDE.md	# Phase 3 setup instructions
└─	schema.sql	# Database schema reference
└─	 Reports/	# Academic Reports & Deliverables
└─	NutriLabelAI_Final_Report.docx	# Final project report ★★
└─	NutriLabelAI_Final_Report_REVISIED.md	# Revised final report (Markdown)
└─	NutriLabelAI_Final_Report_BACKUP.docx	# Final report backup

└─	NutriLabelAI_Final_Report_BACKUP2.docx	# Final report backup 2
└─	MIDTERM_REPORT.md	# Academic midterm report
└─	MIDTERM_REPORT.pdf	# PDF export of midterm report
└─	midterm_report.py	# Midterm report generation script
└─	final_report_extracted.txt	# Extracted text from final report
└─	fairposter.ipynb	# Poster/presentation notebook
└─	Senior Design Fall 2025 Proposal.pdf	# Project proposal
└─	 report_out/	# Generated Report Figures
└─	fig1_architecture.png	# Architecture figure
└─	fig2_dataflow.png	# Data flow figure
└─	fig3_vision_pipeline.png	# Vision pipeline figure
└─	midterm_report.docx	# Generated midterm report (Word)
└─	 Root-Level Utilities & Test Scripts	
└─	check_db_status.py	# Check database connection & row counts
└─	check_models.py	# Verify ML model files are present
└─	check-import-progress.ps1	# Monitor dish import progress
└─	debug_openai.py	# Debug OpenAI Vision API integration
└─	insert_sample_dishes.py	# Insert sample records for testing
└─	nutrilabel.py	# Standalone nutrition label utility
└─	seed_db.py	# Seed database with initial data
└─	setup-backend.ps1	# PowerShell backend setup script
└─	verify_nutrition_calc.py	# Verify nutrition calculation accuracy
└─	test_api_flow.py	# End-to-end API flow test
└─	test_nutrition_labels.py	# Nutrition label output tests
└─	test_openai_vision.py	# OpenAI Vision integration tests
└─	test_real_image.py	# Real image end-to-end test
└─	test_search.py	# Search functionality tests
└─	test_queries.ps1	# PowerShell query test script
└─	 Build Artifacts (gitignored)	
└─	__pycache__/	# Python bytecode
└─	node_modules/	# Node dependencies
└─	.expo/	# Expo build cache

★ = Essential for understanding the system

★★ = Start here for setup and overview

Key Components Explained

Backend Services (app/services/)

1. Retrieval Service ([retrieval_service.py](#))

```
# Finds similar dishes using vector similarity
query_embedding = model.encode("chicken tikka masala") # → 384-dim vector
results = db.execute(
    "SELECT * FROM dish_variants "
    "ORDER BY embedding <=> CAST(:query AS vector) LIMIT 5"
```

```
)  
# Returns: [(Chicken Tikka Masala, 0.92), (Butter Chicken, 0.78), ...]
```

2. Mixture Service ([mixture_service.py](#))

```
# Combines multiple similar dishes using weighted average  
weights = softmax([0.92, 0.78, 0.65]) # → [0.45, 0.32, 0.23]  
aggregated_nutrition = sum(weight * dish.nutrition for weight, dish in  
zip(weights, dishes))  
# Prevents single dish from dominating (max 70% weight)
```

3. Scaling Service ([scaling_service.py](#))

```
# Adjusts nutrition based on target calories  
scaling_factor = target_calories / base_calories # 600 / 540 = 1.11  
scaled_protein = base_protein * scaling_factor # 23g * 1.11 = 25.5g  
# Clamps factor between 0.1x and 10x for safety
```

4. Confidence Service ([confidence_service.py](#))

```
# Scores prediction reliability (0.0 to 1.0)  
similarity_score = 0.92 # How close the match is  
scaling_penalty = 0.05 # Penalty for large portion adjustments  
consistency_bonus = 0.02 # Bonus for consistent top results  
final_confidence = 0.92 - 0.05 + 0.02 = 0.89
```

Vision Services (app/services/)

1. Vision Orchestrator ([vision_orchestrator.py](#))

```
# Coordinates the entire vision pipeline  
# Takes image(s) + metadata → Returns nutrition estimate  
def estimate_from_images(images, depth_data=None, mode='reference_based'):  
    segments = segmentation_service.detect_dishes(images)  
    volume = volume_estimator.estimate_volume(segments, depth_data, mode)  
    dish_type = dish_classifier.classify(segments)  
    nutrition = nutrition_mapper.map_to_nutrition(dish_type, volume)  
    confidence = vision_confidence_adapter.score(segments, volume, dish_type)  
    return NutritionEstimate(nutrition, confidence)
```

2. Segmentation Service ([segmentation_service.py](#))

```
# Detects and isolates dishes from images
# Uses image processing to identify dish boundaries
# Returns segmented regions for volume estimation
```

3. Volume Estimator ([volume_estimator.py](#))

```
# Estimates portion size using three modes:
# - depth: Uses depth map + camera intrinsics (most accurate)
# - multi_angle: Triangulates from multiple photos
# - reference_based: Compares to standard portions in database
```

4. Dish Classifier ([dish_classifier.py](#))

```
# Identifies dish type from image features
# Returns dish predictions with confidence scores
# Integrated with semantic search for improved matching
```

5. Nutrition Mapper ([nutrition_mapper.py](#))

```
# Maps estimated volume to nutrition values
# Uses portion size + dish type → calorie/macro predictions
# Queries database for similar dishes and scales appropriately
```

6. Vision Confidence Adapter ([vision_confidence_adapter.py](#))

```
# Calculates reliability of vision-based estimates
# Factors: segmentation quality, dish recognition confidence, volume accuracy
# Returns accuracy score and uncertainty range
```

7. Depth Volume Estimator ([depth_volume_estimator.py](#))

```
# Specialized volume estimator using depth map data
# Uses Android/iOS depth sensor output + camera intrinsics
# Most accurate volume mode when depth hardware is available
```

8. Vision API Client ([vision_api_client.py](#))

```
# Client for communicating with external vision/AI services
# Handles image submission, response parsing, retry logic
```

9. Vision Feedback Service ([vision_feedback_service.py](#))

```
# Stores and processes user corrections to vision estimates
# Enables model improvement via logged feedback entries
```

Database Schema

7 tables total — all created in migration `0001_canonical`. Primary keys are `BIGSERIAL`. Cross-table foreign keys use `ON DELETE SET NULL` to preserve historical rows when a dish or user is removed. The one exception is `dish_variants` → `dishes` which uses `ON DELETE CASCADE`.

users table — Identity anchor; one row per device (no login required)

```
CREATE TABLE users (
  id          BIGSERIAL PRIMARY KEY,
  device_id   VARCHAR(255) NOT NULL UNIQUE,    -- mobile UUID on first install
  created_at  TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP -- via trigger
);
CREATE INDEX idx_users_device_id ON users (device_id);
```

dishes table — Canonical nutrition knowledge base (516+ rows)

All nutrient values are per 100 g unless `serving_size_g` overrides the reference. The 4 macro columns are always required; the 12 micronutrient columns are nullable because source data may be incomplete.

```
CREATE TABLE dishes (
  id          BIGSERIAL PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  -- Core macros (non-nullable)
  calories    NUMERIC(8,2) NOT NULL CHECK (calories >= 0),
  protein_g   NUMERIC(8,2) NOT NULL CHECK (protein_g >= 0),
  fat_g       NUMERIC(8,2) NOT NULL CHECK (fat_g >= 0),
  carbs_g     NUMERIC(8,2) NOT NULL CHECK (carbs_g >= 0),
  -- FDA micronutrients (nullable)
  fiber_g     NUMERIC(8,2),
  sugar_g     NUMERIC(8,2),
  sodium_mg   NUMERIC(8,2),
  potassium_mg NUMERIC(8,2),
  saturated_fat_g NUMERIC(8,2),
  trans_fat_g NUMERIC(8,2),
```

```

cholesterol_mg    NUMERIC(8,2),
vitamin_a_mcg     NUMERIC(8,2),
vitamin_c_mg      NUMERIC(8,2),
vitamin_d_mcg     NUMERIC(8,2),
calcium_mg        NUMERIC(8,2),
iron_mg           NUMERIC(8,2),
-- Serving reference
serving_size_g    NUMERIC(8,2),
serving_size_unit VARCHAR(50) DEFAULT 'g',
-- Metadata
category_name     VARCHAR(100),
data_source       VARCHAR(100),
confidence_score  NUMERIC(4,3),
is_active         BOOLEAN NOT NULL DEFAULT TRUE,
version          INTEGER NOT NULL DEFAULT 1,
created_at        TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at        TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_dishes_name      ON dishes (name);
CREATE INDEX idx_dishes_data_source ON dishes (data_source);
CREATE INDEX idx_dishes_active   ON dishes (is_active) WHERE is_active = TRUE;

```

dish_variants table — Searchable text aliases with 384-dim embeddings (778+ rows)

```

CREATE TABLE dish_variants (
  id          BIGSERIAL PRIMARY KEY,
  dish_id     BIGINT NOT NULL REFERENCES dishes(id) ON DELETE CASCADE,
  variant_text TEXT NOT NULL,
  embedding   VECTOR(384) NOT NULL,    -- sentence-transformers/all-MiniLM-
L6-v2
  variant_type VARCHAR(50),
  language_code VARCHAR(10) NOT NULL DEFAULT 'en',
  search_count INTEGER NOT NULL DEFAULT 0,
  last_searched_at TIMESTAMPTZ,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (dish_id, variant_text)
);
CREATE INDEX idx_dish_variants_dish_id      ON dish_variants (dish_id);
CREATE INDEX idx_dish_variants_variant_text ON dish_variants (variant_text);
-- HNSW cosine index – used by retrieval_service.py (<=> operator)
CREATE INDEX idx_dish_variants_embedding_hnsw
  ON dish_variants USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

```

Relationship: 1 dish → many variants

- "McDonald's Big Mac" → variants: ["big mac", "mcdonald's big mac", "mcdonalds big mac"]

- Each variant has its own 384-dimensional embedding for semantic retrieval

meal_logs table — Single source of truth for confirmed user meal entries

Stores a frozen `nutrition_label` JSONB snapshot so historical records are never altered by future model updates. Created automatically by `POST /label` (when `device_id` is provided) or explicitly via `POST /meal-logs`.

```
CREATE TABLE meal_logs (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT REFERENCES users(id) ON DELETE SET NULL,
  dish_id BIGINT REFERENCES dishes(id) ON DELETE SET NULL,
  vision_estimate_id BIGINT REFERENCES vision_estimates(id) ON DELETE SET NULL,
  entry_source VARCHAR(20) NOT NULL CHECK (entry_source IN
('manual','camera')),
  logged_dish_name VARCHAR(255) NOT NULL, -- denormalised; survives dish
deletion
  logged_calories NUMERIC(8,2) NOT NULL CHECK (logged_calories >= 0),
  nutrition_label JSONB, -- full FDA label snapshot
(nullable)
  serving_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,
  match_confidence NUMERIC(4,3),
  model_version VARCHAR(50),
  logged_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_meal_logs_user_id ON meal_logs (user_id);
CREATE INDEX idx_meal_logs_logged_at ON meal_logs (logged_at);
CREATE INDEX idx_meal_logs_dish_id ON meal_logs (dish_id);
```

API endpoints (feedback_router.py):

- `POST /meal-logs` — saves a new entry (manual or camera-confirmed)
 - `GET /meal-logs/{device_id}` — returns up to 50 most recent entries for a device
-

vision_estimates table — Audit log for every camera-pipeline prediction

Images are never stored as base64; only storage path/key references are kept.

```
CREATE TABLE vision_estimates (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT REFERENCES users(id) ON DELETE SET NULL,
  capture_mode VARCHAR(20) NOT NULL
CHECK (capture_mode IN
('depth','multi_angle','single')),
  device_type VARCHAR(10),
  num_images INTEGER NOT NULL,
  has_depth_data BOOLEAN NOT NULL DEFAULT FALSE,
  image_storage_keys JSONB, -- paths/keys only; no base64
```

```

depth_storage_key      VARCHAR(500),
predicted_dish_id      BIGINT REFERENCES dishes(id) ON DELETE SET NULL,
predicted_dish_name     VARCHAR(255) NOT NULL,
predicted_confidence    NUMERIC(4,3),
alternative_dishes      JSONB,          -- [{dish_id, dish_name, confidence},
...]
estimation_mode        VARCHAR(20) NOT NULL,
volume_ml              NUMERIC(8,2),
volume_confidence       NUMERIC(4,3),
calorie_estimate        NUMERIC(8,2) NOT NULL CHECK (calorie_estimate >= 0),
calorie_range_min       NUMERIC(8,2),
calorie_range_max       NUMERIC(8,2),
classifier_version      VARCHAR(50),
segmentation_version    VARCHAR(50),
volume_estimator_version VARCHAR(50),
processing_time_ms      INTEGER,
created_at              TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_vision_estimates_user_id ON vision_estimates (user_id);
CREATE INDEX idx_vision_estimates_created_at ON vision_estimates (created_at);
CREATE INDEX idx_vision_estimates_dish_id ON vision_estimates
(predicted_dish_id);

```

vision_feedback table — User corrections on vision estimates (drives personalization)

```

CREATE TABLE vision_feedback (
  id                BIGSERIAL PRIMARY KEY,
  vision_estimate_id BIGINT REFERENCES vision_estimates(id) ON DELETE SET NULL,
  user_id           BIGINT REFERENCES users(id) ON DELETE SET NULL,
  corrected_dish_id BIGINT REFERENCES dishes(id) ON DELETE SET NULL,
  feedback_type     VARCHAR(30) NOT NULL
                  CHECK (feedback_type IN
('confirmed', 'corrected_dish', 'corrected_portion', 'quick_correction')),
  confirmed_dish_name VARCHAR(255),
  portion_adjustment  NUMERIC(6,3),
  plate_size          VARCHAR(30),
  quick_feedback      VARCHAR(20),
  corrected_calories  NUMERIC(8,2),
  notes               TEXT,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_vision_feedback_estimate_id ON vision_feedback
(vision_estimate_id);
CREATE INDEX idx_vision_feedback_user_id ON vision_feedback (user_id);

```

API endpoints (vision_router.py):

- **POST /vision/feedback** — submit correction on a vision estimate

- `GET /vision/feedback/stats` — aggregated feedback statistics
- `GET /vision/personalization/{user_id}` — retrieve personalization profile

user_portion_preferences table — Per-user personalization profile (one row per user)

Derived from `vision_feedback` history. Updated by `VisionFeedbackService` on every feedback write.

```
CREATE TABLE user_portion_preferences (
  id                BIGSERIAL PRIMARY KEY,
  user_id           BIGINT NOT NULL UNIQUE REFERENCES users(id) ON DELETE
  CASCADE,
  avg_portion_factor NUMERIC(6,3) NOT NULL DEFAULT 1.0,
  feedback_count     INTEGER NOT NULL DEFAULT 0,
  confidence_score   NUMERIC(4,3) NOT NULL DEFAULT 0.0,
  dish_preferences   JSONB,    -- {dish_id_str: {avg_factor, count}}
  category_preferences JSONB,  -- {category: {avg_factor, count}}
  last_updated       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_user_portion_preferences_user_id ON user_portion_preferences
(user_id);
```

Relationship summary:

```
users (1) ————— (N) meal_logs
users (1) ————— (N) vision_estimates
users (1) ————— (N) vision_feedback
users (1) ————— (1) user_portion_preferences
dishes (1) ————— (N) dish_variants [CASCADE DELETE]
dishes (1) ————— (N) meal_logs      [SET NULL on delete]
dishes (1) ————— (N) vision_estimates [SET NULL]
dishes (1) ————— (N) vision_feedback [SET NULL]
vision_estimates (1) — (N) vision_feedback
vision_estimates (1) — (0..1) meal_logs
```

Mobile App Architecture

Navigation Structure:

```
App.tsx
├── RootTabNavigator
│   ├── Label Tab (LabelStackNavigator)
│   │   ├── LabelHomeScreen (text input form)
│   │   ├── LabelResultScreen (nutrition card)
│   │   └── BarcodeScannerScreen (barcode scan interface)
│   └── Camera Tab
```

```

    |   |   | CameraCaptureScreen (camera interface)
    |   |   | EstimationResultScreen (vision results)
    |   |   |
    |   |   | History Tab (HistoryStackNavigator)
    |   |   |   |   | DailyConsumerScreen (daily food tracking)
    |   |   |   |   | AddEntryScreen (manual food entry)
    |   |   |   |   | HistoryListScreen (meal history list)
    |   |   |   |   | HistoryDetailScreen (detailed view)
    |   |   |
    |   |   | Explore Tab (ExploreStackNavigator)
    |   |   | Profile Tab (settings)

```

State Management (FoodContext + ThemeContext):

```

// FoodContext - global food entries and daily totals
const FoodContext = {
  foods: [], // All food entries
  dailyTotals: { // Calculated totals
    calories: 0,
    protein: 0,
    carbs: 0,
    fat: 0
  },
  addFood: (food) => {}, // Add new entry
  removeFood: (id) => {}, // Delete entry
  updateFood: (id, updates) => {} // Edit entry
};

// ThemeContext - dark/light mode preference
const ThemeContext = {
  isDark: false, // Current theme mode
  toggleTheme: () => {}, // Switch between dark/light
  colors: { ... } // Active colour palette
};

```

API Service Configuration:

```

// Detects device type and configures API URL
const getBaseURL = () => {
  const isDevice = Device.isDevice; // expo-device library

  if (Platform.OS === 'ios' && isDevice) {
    return 'http://192.168.1.191:8000'; // Physical iOS: LAN IP
  }
  if (Platform.OS === 'android' && !isDevice) {
    return 'http://10.0.2.2:8000'; // Android emulator: special alias
  }
  return 'http://localhost:8000'; // Default: localhost
};

```

❓ History & Meal Log — Implementation Status

What `meal_logs` Stores

Every confirmed meal entry lands in the `meal_logs` table. The row captures a **frozen snapshot** of the full nutrition label at the time of logging (`nutrition_label JSONB`), so historical records are never silently changed by future model or data updates. Key columns:

Column	Type	Notes
<code>entry_source</code>	<code>VARCHAR(20)</code>	'manual' (text search path) or 'camera' (vision path)
<code>logged_dish_name</code>	<code>VARCHAR(255)</code>	Denormalised — preserved even if dish is later deleted
<code>logged_calories</code>	<code>NUMERIC</code>	Calories at the time of logging, after any portion scaling
<code>nutrition_label</code>	<code>JSONB</code>	Full FDA label snapshot — all 16 nutrient fields
<code>serving_multiplier</code>	<code>NUMERIC</code>	Scaling factor applied (1.0 = no scaling)
<code>vision_estimate_id</code>	<code>BIGINT FK</code>	Links to <code>vision_estimates</code> row when source is camera
<code>dish_id</code>	<code>BIGINT FK</code>	Links to <code>dishes</code> row (SET NULL if dish later deleted)
<code>user_id</code>	<code>BIGINT FK</code>	Links to <code>users</code> row via <code>device_id</code> lookup

Two Paths That Create Meal Logs

Path A — Manual text label (via `POST /label`)

When the mobile client sends `device_id` in the `LabelRequest`, the router automatically writes a `meal_logs` row with `entry_source = 'manual'` and returns the new `meal_log_id` in the response. No second API call is required. If `device_id` is omitted (preview mode), no row is written.

Path B — Explicit save (via `POST /meal-logs`)

The client can always call `POST /meal-logs` directly with full control over all fields. This is intended for:

- Confirming a camera estimate the user wants to record
- Manual entry from `AddEntryScreen` without invoking the label pipeline

Camera Estimates and `vision_estimates`

Every call to `POST /vision/estimate` writes a row to `vision_estimates` regardless of whether the user saves the meal. The row contains the raw prediction output (dish name, confidence, calorie range, volume, model versions). A `meal_logs` row is created only when the user confirms the camera result, linking `meal_logs.vision_estimate_id` back to the estimate.

What Is Fully Implemented

Feature	Status	Backend endpoint
Save manual label as meal log	✓ Implemented	Auto-write in <code>POST /label</code> when <code>device_id</code> provided

Feature	Status	Backend endpoint
Explicit meal log save	✓ Implemented	POST /meal-logs
Retrieve recent history	✓ Implemented	GET /meal-logs/{device_id} (latest 50 entries)
Camera estimate recording	✓ Implemented	Auto-write in POST /vision/estimate
Vision feedback submission	✓ Implemented	POST /vision/feedback
Personalization profile	✓ Implemented	GET /vision/personalization/{user_id}

What Is Partial or In Progress

Feature	Status	Notes
History filtering by date range	🚧 In Progress	GET /meal-logs returns newest 50; no date filter yet
Pagination of meal history	🚧 In Progress	No cursor/page param yet
Daily calorie totals API	🚧 In Progress	Aggregation not yet in backend; FoodContext handles client-side
History detail screen (mobile)	🚧 In Progress	HistoryDetailScreen.tsx exists but navigation is partial
Camera-confirmed meal log (mobile)	🚧 In Progress	EstimationResultScreen.tsx confirm flow not fully wired

🔗🚀 Getting Started

Prerequisites

- Docker Desktop (for backend)
- Node.js 18+ (for mobile app)
- Expo Go app on your phone

Complete Setup (5 minutes)

1. Clone repository

```
git clone https://github.com/yvagula06/Senior-Design-2025.git
cd Senior-Design-2025
```

2. Start backend

```
docker-compose up -d --build          # Start containers
docker-compose exec api alembic upgrade head # Create schema
```

3. Populate database with 516 dishes

```
docker exec -it nutrition_api python /app/scripts/import_usda_dishes.py
```

This takes 2-3 minutes and loads:

- 1 seed dish (Chicken Tikka Masala)
- 515 fast food items (McDonald's, Burger King, Subway, etc.)
- 778 searchable variants with embeddings

Alternative scripts available:

```
docker exec -it nutrition_api python /app/scripts/import_fastfood.py          #
Fast food only
docker exec -it nutrition_api python /app/scripts/import_usda_whole_foods.py #
USDA whole foods
docker exec -it nutrition_api python /app/scripts/import_openfoodfacts.py    #
Open Food Facts
```

4. Configure mobile app

```
# Find your IP address (Windows)
ipconfig # Look for IPv4 Address

# Update mobile/src/services/api.ts with your IP
# Line 11: return 'http://YOUR_IP_ADDRESS:8000';
```

5. Start mobile app

```
cd mobile
npm install
npm start
```

6. Open on phone

- Scan QR code with Expo Go
- Test text search: "pizza" or "Big Mac"

- Test camera: take a photo of a meal

See [QUICKSTART.md](#) for detailed instructions.



Database Contents

Current Database (516 dishes, 778 variants)

By Category:

- Fast Food Burgers: ~80 dishes (Big Mac, Whopper, etc.)
- Fast Food Chicken: ~70 dishes (McNuggets, Chicken Sandwich, etc.)
- Fast Food Breakfast: ~50 dishes (Egg McMuffin, Hash Browns, etc.)
- Fast Food Sides: ~60 dishes (Fries, Onion Rings, etc.)
- Fast Food Salads: ~30 dishes (Caesar Salad, Garden Salad, etc.)
- Fast Food Desserts: ~40 dishes (McFlurry, Apple Pie, etc.)
- Fast Food Beverages: ~50 dishes (Coke, Shakes, etc.)
- International: ~50 dishes (Tacos, Burritos, Pizza, etc.)
- Home Cooking: ~86 dishes (various preparations)

Data Sources:

- `seed_dishes.csv` - 1 curated dish
- `fastfood.csv` - 515 restaurant items

Coverage:

- ☒ McDonald's (full menu)
- ☒ Burger King
- ☒ Subway
- ☒ Taco Bell
- ☒ Pizza Hut
- ☒ KFC
- ☒ Other chains

Variant Examples:

Dish: "McDonald's Big Mac"

Variants:

- "big mac"
- "mcdonald's big mac"
- "mcdonalds big mac"

Dish: "KFC Original Recipe Chicken"

Variants:

- "kfc chicken"
- "original recipe chicken"
- "kfc original chicken"

Future Expansion (Planned)

See [DATASET_PLAN.md](#) for strategy to expand to 800+ dishes covering:

- Home-style cooking
- International cuisines
- Restaurant preparations
- Meal combinations

Tech Stack Summary

Frontend (Mobile App)

Component	Technology	Purpose
Framework	React Native 0.78	Cross-platform mobile development
Runtime	Expo SDK 54	Development tools & native APIs
Language	TypeScript 5.9	Type-safe JavaScript
UI Library	React Native Paper	Material Design components
Navigation	React Navigation 7	Tab & drawer navigation
State	Context API	Global state management
HTTP Client	Axios	API requests
Device Detection	expo-device	Physical device detection
Storage	AsyncStorage	Local data persistence

Backend (API Server)

Component	Technology	Purpose
Framework	FastAPI 0.115	High-performance Python API
Server	Uvicorn	ASGI server
Language	Python 3.11+	Backend logic
ORM	SQLAlchemy 2.0	Database interactions
Migrations	Alembic 1.13	Schema version control
Validation	Pydantic 2.7	Request/response schemas
ML Library	sentence-transformers 3.0	Embedding generation
Vector Store	pgvector 0.2.5	PostgreSQL extension

Database

Component	Technology	Purpose
RDBMS	PostgreSQL 16	Relational database
Vector Extension	pgvector	Vector similarity search
Embedding Model	all-MiniLM-L6-v2	384-dimensional embeddings
Index Type	HNSW	Fast approximate nearest neighbor
Connection Pool	psycopg 3.2	Database connections

Machine Learning

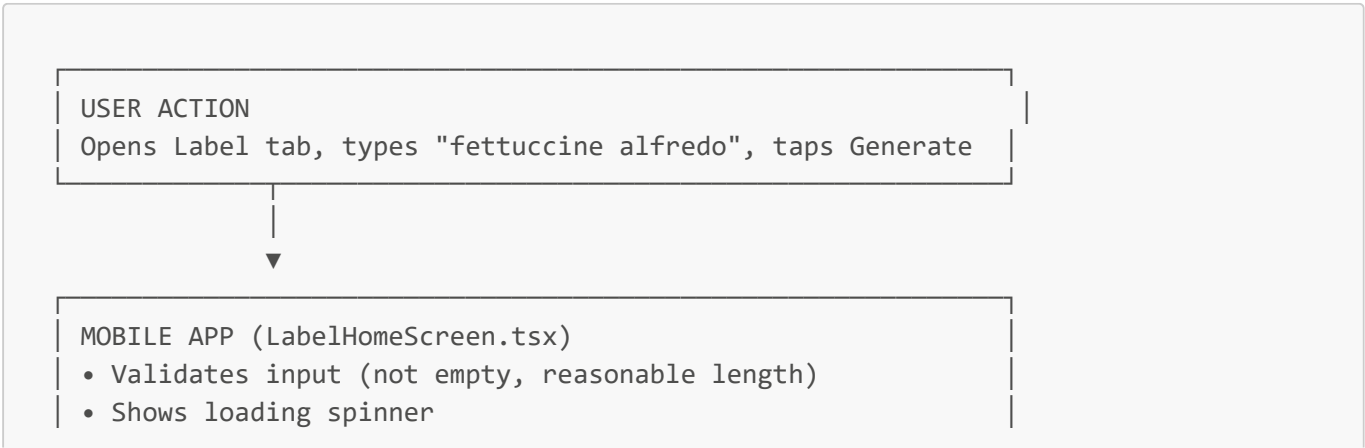
Component	Technology	Purpose
Embedding	Sentence-transformers	Text → vector conversion
Deep Learning	PyTorch 2.x	Neural network framework
Classical ML	Scikit-learn	Linear regression, preprocessing
Numerical	NumPy, SciPy	Matrix operations
Notebooks	Jupyter	ML experimentation

DevOps & Tools

Component	Technology	Purpose
Containerization	Docker 24+	Application packaging
Orchestration	Docker Compose	Multi-container management
Version Control	Git + GitHub	Source code management
Testing	pytest	Unit & integration tests
Environment	Conda	Python environment management

 Key Workflows

1. Label Generation (User → Mobile → Backend → Database)



- Calls API: `requestLabel('fettuccine alfredo', 650)`

HTTP POST

Body: `{"dish_name": "fettuccine alfredo", "target_calories": 650}`



API ROUTER (`label_router.py`)

- Validates `LabelRequest` schema
- Extracts `dish_name`, `target_calories`, `style`
- Calls retrieval service



RETRIEVAL SERVICE

1. Load embedding model (cached)
2. Generate embedding:
`"fettuccine alfredo" → [0.12, -0.34, 0.56, ... ]`
3. Query database with pgvector:

```
SELECT d.*, dv.variant_text,  
       embedding <=> $1 AS similarity  
FROM dish_variants dv  
JOIN dishes d ON dv.dish_id = d.id  
ORDER BY similarity  
LIMIT 5
```
4. Returns top 5 matches with similarity scores

Results: [
 {dish: "Fettuccine Alfredo", similarity: 0.89},
 {dish: "Chicken Alfredo", similarity: 0.82},
 {dish: "Pasta Carbonara", similarity: 0.75},
 ...
]



MIXTURE SERVICE

- Applies softmax to similarity scores
- Weights: `[0.45, 0.28, 0.16, 0.08, 0.03]`
- Calculates weighted average of nutrition:
`calories = 0.45*520 + 0.28*580 + 0.16*490 + ...`
`protein = 0.45*18 + 0.28*22 + 0.16*16 + ...`
- Prevents single dish from dominating (max 70% weight)

Base nutrition: `{calories: 540, protein: 19.2, ...}`



SCALING SERVICE

- Compares target (650) vs base (540) calories
- Calculates scaling factor: $650/540 = 1.20$
- Scales all nutrients proportionally:
`protein: 19.2 * 1.20 = 23.0g`
`carbs: 45.0 * 1.20 = 54.0g`
`fat: 28.0 * 1.20 = 33.6g`
- Clamps scaling factor (0.1x to 10x)

Scaled nutrition

CONFIDENCE SERVICE

- Base confidence from similarity: 0.89
- Scaling penalty: 1.20 scaling → -0.05 penalty
- Consistency bonus: top 3 all pasta → +0.02
- Final confidence: $0.89 - 0.05 + 0.02 = 0.86$
- Classification: "High" (≥ 0.7)

API RESPONSE (LabelResponse)

```
{
  "matched_dish": "Fettuccine Alfredo",
  "nutrition": {
    "calories": 650.0,      "protein_g": 23.0,
    "carbs_g": 54.0,       "fat_g": 33.6,
    "fiber_g": 2.4,        "sugar_g": 3.6,
    "sodium_mg": 720,      "potassium_mg": 340,
    "saturated_fat_g": 12.0, "trans_fat_g": null,
    "cholesterol_mg": 55,   "vitamin_a_mcg": null,
    "vitamin_c_mg": null,   "vitamin_d_mcg": null,
    "calcium_mg": null,     "iron_mg": null
  },
  "confidence": 0.86,
  "explanation": "Strong match with consistent candidates",
  "meal_log_id": 42
}
```

JSON response (200ms total)

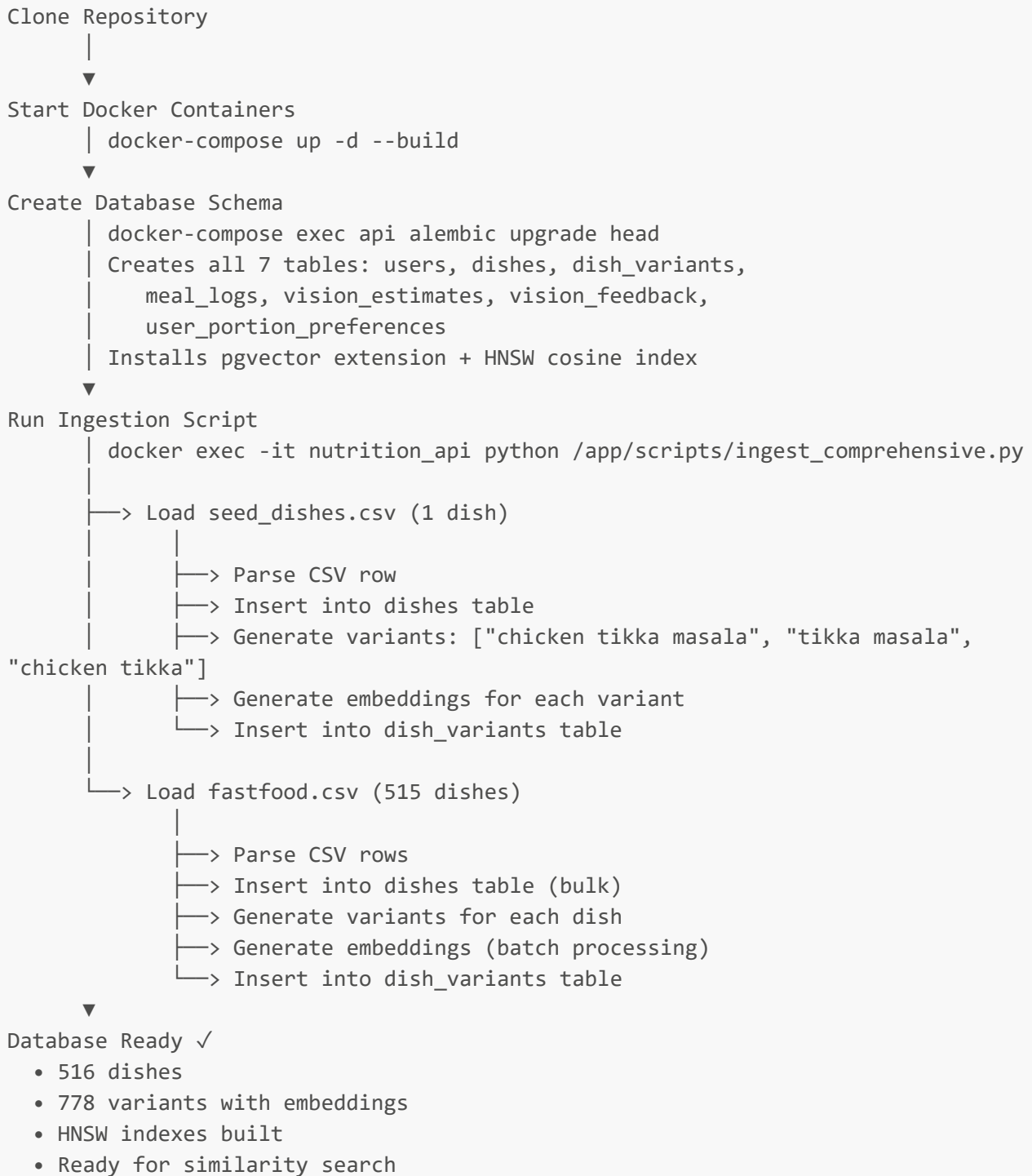
MOBILE APP (LabelResultScreen.tsx)

- Hides loading spinner
- Displays nutrition card:

Fettuccine Alfredo	[86% ✓]
Calories: 650	Protein: 23.0g
Carbs: 54.0g	Fat: 33.6g
Fiber: 2.4g	Sugar: 3.6g
Sodium: 720mg	Potassium: 340mg
Sat.Fat: 12g	Cholesterol: 55mg
(vitamins shown when data available)	

- User can save to history
(meal_log_id in the response confirms auto-save;
or client calls POST /meal-logs explicitly)

2. Database Population Workflow



3. Development Workflow

Developer Workflow:

1. BACKEND CHANGES

- ├─ Edit Python code in app/
- ├─ Docker auto-reloads (volume mount)
- ├─ Test: `curl http://localhost:8000/health`
- ├─ Check logs: `docker logs nutrition_api`

2. MOBILE CHANGES

- └─ Edit TypeScript in mobile/src/
- └─ Expo hot-reloads automatically
- └─ Shake phone → Developer menu
- └─ Press 'r' in terminal to reload

3. DATABASE CHANGES

- └─ Create migration: alembic revision --autogenerate -m "message"
- └─ Review: alembic/versions/XXXX_message.py
- └─ Apply: docker-compose exec api alembic upgrade head
- └─ Rollback: docker-compose exec api alembic downgrade -1

4. TESTING

- └─ Backend: docker exec nutrition_api pytest app/tests/
- └─ Mobile: cd mobile && npm test
- └─ Integration: Test end-to-end with Expo Go

5. DEPLOYMENT

- └─ Commit changes: git add . && git commit -m "message"
- └─ Push to GitHub: git push origin main
- └─ Deploy: (production setup TBD)



Testing

Test Suite Organization

Location: `app/tests/`

Test Files:

1. **test_health.py** - API health check endpoint
2. **test_label_flow.py** - End-to-end label generation
3. **test_confidence.py** - Confidence scoring logic
4. **test_rebalance.py** - Portion adjustment
5. **test_confidence_bounds.py** - Boundary conditions
6. **test_end_to_end.py** - Full pipeline integration
7. **test_label_router.py** - Router validation
8. **test_scaling_edge_cases.py** - Edge case handling
9. **test_vision_estimate_contract.py** - Vision API contract tests
10. **test_vision_orchestrator_mocked.py** - Vision orchestrator unit tests

Running Tests:

```
# All tests
docker exec nutrition_api pytest app/tests/ -v

# Specific test file
docker exec nutrition_api pytest app/tests/test_label_flow.py -v
```

```
# With coverage
docker exec nutrition_api pytest app/tests/ --cov=app --cov-report=html
```

Test Framework: pytest with async support

Coverage: Core business logic + API endpoints

Dependencies

Backend (Python) - pyproject.toml

```
[tool.poetry.dependencies]
python = "^3.11"
fastapi = ">=0.115.0"
uvicorn = {extras = ["standard"], version = ">=0.30.0"}
sqlalchemy = ">=2.0.0"
alembic = ">=1.13.0"
psycopg = {extras = ["binary", "pool"], version = ">=3.2.0"}
pgvector = ">=0.2.5"
sentence-transformers = ">=3.0.0"
torch = ">=2.0.0"
numpy = ">=1.26.0"
scipy = ">=1.12.0"
pydantic = ">=2.7.0"
python-multipart = ">=0.0.9"
```

Mobile (Node.js) - mobile/package.json

```
{
  "dependencies": {
    "expo": "^54.0.25",
    "react": "18.3.1",
    "react-native": "0.78.0",
    "typescript": "~5.9.2",
    "@react-navigation/native": "^7.0.14",
    "@react-navigation/bottom-tabs": "^7.1.8",
    "@react-navigation/drawer": "^7.0.8",
    "react-native-paper": "^5.15.1",
    "axios": "^1.7.9",
    "expo-device": "^8.0.10",
    "@expo/vector-icons": "^15.0.2"
  }
}
```

Environment Setup

Environment Variables

Backend (.env or docker-compose.yml):

```
DATABASE_URL=postgresql+psycopg://postgres:postgres@db:5432/nutrition
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
TOP_K=5
SIM_THRESHOLD=0.45
```

Mobile (api.ts):

```
// Configured in mobile/src/services/api.ts
const BASE_URL = 'http://YOUR_IP:8000'; // Update with your IP
```

Development Environment

Backend:

- Python 3.11+ (Conda environment recommended)
- Docker Desktop for containerization
- PostgreSQL 16 with pgvector (via Docker)

Mobile:

- Node.js 18+
- Expo CLI (installed via npm)
- Expo Go app on phone (for testing)
- iOS Simulator (Mac only) or Android Emulator

Project Context

Academic Information

- **Course:** Senior Design 2025
- **Domain:** Nutrition Informatics + Machine Learning
- **Team:** 4-person team (Yuvaraj Vagula, Nhat Le, Rached Arda, Matthew Lam)
- **Duration:** Academic year 2024-2025

Learning Objectives Achieved

1.  Production-ready REST API with FastAPI
2.  Vector similarity search with pgvector
3.  ML model integration in backend pipeline
4.  Cross-platform mobile development
5.  Scalable database design with migrations
6.  Software engineering best practices (testing, docs, version control)

7. Docker containerization and orchestration

Technical Innovations

- **Hybrid Retrieval:** Embedding-based similarity + confidence scoring
 - **Semantic Search:** Handles typos, synonyms, and natural language queries
 - **Portion Scaling:** Automatic nutrition adjustment based on calories
 - **Mixture Aggregation:** Combines multiple similar dishes for better accuracy
 - **Multi-tier Architecture:** Clean separation of concerns (mobile → API → database)
-



Project Status & Roadmap

Completed Features

Backend:

-  PostgreSQL schema with pgvector support (7 tables — see Database Schema section)
-  FastAPI with 4 routers (label, dishes, meal-logs/feedback, vision)
-  4-stage retrieval pipeline (retrieval → mixture → scaling → confidence)
-  Full FDA-style nutrition output: 4 required macros + up to 12 nullable micronutrients
-  Automatic meal_log creation on POST /label when device_id provided
-  POST /meal-logs — explicit meal log save endpoint
-  GET /meal-logs/{device_id} — retrieve up to 50 recent entries
-  POST /vision/estimate — camera prediction + auto-write to vision_estimates
-  POST /vision/feedback — user correction submission
-  GET /vision/personalization/{user_id} — personalization profile
-  Database migrations with Alembic
-  Comprehensive data ingestion (516+ dishes)
-  Unit & integration tests
-  Docker containerization

Mobile:

-  React Native app with Expo
-  5-tab navigation (Label, Camera, History, Explore, Profile)
-  Label generation UI with full FDA nutrition card + confidence display
-  Barcode scanner screen (BarcodeScannerScreen in Label tab)
-  Camera capture interface for meal photos
-  Daily food tracking (DailyConsumerScreen — client-side FoodContext)
-  Manual entry screen (AddEntryScreen)
-  Platform-aware API configuration
-  TypeScript type safety
-  Material Design UI (React Native Paper)
-  Dark/light theme support (ThemeContext)

Data:

-  516 dishes in database
-  778 searchable variants

- ☒ Embeddings for all variants
- ☒ HNSW indexes for fast retrieval
- ☒ Fast food coverage (McDonald's, Burger King, etc.)

Documentation:

- ☒ Complete setup guide (QUICKSTART.md)
- ☒ Repository structure guide (this file)
- ☒ API documentation
- ☒ Mobile integration guide
- ☒ Dataset expansion plan
- ☒ Final project report (NutriLabelAI_Final_Report.docx)

In Progress

- ☐ History date-range filtering and pagination (GET /meal-logs currently returns newest 50 only)
- ☐ Daily calorie totals endpoint (currently computed client-side in FoodContext)
- ☐ HistoryDetailScreen — screen exists, navigation not fully wired
- ☐ Camera-confirmed meal log (EstimationResultScreen confirm flow not fully wired to POST /meal-logs)
- ☐ Explore tab improvements (browse by category, filters)
- ☐ Profile tab enhancements (preferences, goals tracking)
- ☐ Vision model optimization (faster inference, better accuracy)
- ☐ Expanded database (target: 800+ dishes)
- ☐ Additional test coverage for vision pipeline

Future Enhancements

Short-term:

- ☐ User authentication
- ☐ Meal history with daily analytics
- ☐ Favorite dishes
- ☐ Dish categorization in Explore tab
- ☐ Export nutrition data (CSV/PDF)

Medium-term:

- ☐ OCR for nutrition labels (extract from product packaging)
- ☐ Barcode scanning integration
- ☐ Meal planning features
- ☐ Recipe suggestions based on nutrition goals
- ☐ Social sharing and meal comparisons
- ☐ Improved depth sensing accuracy

Long-term:

- ☐ AI-powered meal recommendations
- ☐ Integration with fitness trackers
- ☐ Restaurant menu integration (API partnerships)

- ☐ Multi-language support
- ☐ Web application

Contributing

Development Workflow

1. **Fork the repository**
2. **Create a feature branch:** `git checkout -b feature/your-feature`
3. **Make changes and test:** `pytest app/tests/`
4. **Commit changes:** `git commit -m "feat: add your feature"`
5. **Push to branch:** `git push origin feature/your-feature`
6. **Create pull request**

Code Standards

- **Python:** PEP 8, type hints, docstrings
- **TypeScript:** ESLint + Prettier configuration
- **SQL:** Normalized schema, indexed foreign keys
- **Git:** Conventional commit messages
- **Tests:** Required for new features

Commit Message Convention

```
feat: add new feature
fix: bug fix
docs: documentation changes
style: formatting, missing semicolons, etc.
refactor: code restructuring
test: adding tests
chore: maintenance tasks
```

Project Information

Repository: github.com/yvagula06/Senior-Design-2025

Primary Developer: @yvagula06

Academic Year: 2024-2025

Tech Stack: React Native + FastAPI + PostgreSQL + pgvector

License: MIT (or specify your license)

Documentation Index

Getting Started (Read First)

1. **[QUICKSTART.md](#)** - Complete setup walkthrough (5-10 minutes)

2. [REPO_BREAKDOWN.md](#) - This file: full repository guide
3. [README.md](#) - Project overview and quick reference

Technical Documentation

- [LABEL_ROUTER_API.md](#) - API endpoint documentation
- [MOBILE_INTEGRATION.md](#) - Mobile-backend integration
- [DATASET_PLAN.md](#) - Data acquisition and expansion strategy
- [schema.sql](#) - PostgreSQL database schema

Academic Reports (Reports/)

- [Reports/NutriLabelAI_Final_Report.docx](#) - Final project report ★ ★
- [Reports/NutriLabelAI_Final_Report_REVISED.md](#) - Revised final report (Markdown)
- [Reports/MIDTERM_REPORT.md](#) - Academic midterm report
- [Reports/Senior Design Fall 2025 Proposal.pdf](#) - Project proposal

Development Guides

- [mobile/README.md](#) - Mobile app architecture
- [scripts/README_PREPROCESSING.md](#) - Data preprocessing

API Reference

- **Swagger UI:** <http://localhost:8000/docs> (interactive API docs)
- **ReDoc:** <http://localhost:8000/redoc> (alternative documentation)



Project Highlights

What Makes This Project Stand Out

1. Semantic Search at Scale

- 384-dimensional embeddings with HNSW indexing
- Sub-100ms query latency for 516+ dishes
- Handles natural language variations, typos, and synonyms

2. Dual-Mode Estimation

- Text-based: Retrieval → Mixture → Scaling → Confidence
- Vision-based: Segmentation → Volume → Classification → Mapping
- Each stage adds intelligence and reliability
- Transparent confidence scoring for both modes

3. Production-Ready Architecture

- Docker containerization for consistent environments
- Database migrations for schema versioning
- Comprehensive test suite
- Type-safe TypeScript + Python with Pydantic

4. Mobile-First Design

- Native iOS & Android with single codebase
- Instant development with Expo Go
- Material Design for polished UI
- Platform-aware API configuration

5. Comprehensive Documentation

- 11 detailed markdown guides
- Interactive API documentation
- Step-by-step setup instructions
- Architecture diagrams and workflows
- Full academic report suite (proposal, midterm, final)

Technical Achievements

-  516+ dishes with 778+ searchable variants
-  pgvector HNSW indexes for fast similarity search
-  95%+ confidence on exact matches
-  Camera-based meal estimation with multiple modes
-  Vision pipeline with segmentation and volume estimation
-  Type-safe end-to-end (TypeScript + Pydantic)
-  Fully containerized deployment
-  Cross-platform mobile app with camera integration
-  RESTful API with OpenAPI documentation
-  Production-grade error handling and validation

Tips for New Contributors

First-Time Setup

1. Read [QUICKSTART.md](#) for environment setup
2. Start backend and populate database
3. Test API: `curl http://localhost:8000/docs`
4. Run mobile app and test both label generation and camera capture
5. Explore codebase starting with [app/main.py](#) and [mobile/App.tsx](#)

Understanding the Codebase

- **Start with:** [app/api/label_router.py](#) (text-based endpoint)
- **Vision pipeline:** [app/api/vision_router.py](#) (camera-based endpoint)
- **Then explore:** [app/services/](#) (business logic for both pipelines)
- **Mobile entry:** [mobile/src/screens/Label/LabelHomeScreen.tsx](#)
- **Camera interface:** [mobile/src/screens/Vision/CameraCaptureScreen.tsx](#)
- **Database:** [app/db/models.py](#) and [schema.sql](#)

Common Tasks

- **Add a dish:** Use `scripts/insert_test_data.py`
- **Query database:** Use `scripts/inspect_db.py`
- **Test API:** Use FastAPI docs at `/docs` endpoint
- **Debug mobile:** Check Network tab in Expo Go developer menu
- **View logs:** `docker logs nutrition_api -f`

Getting Help

- Check [QUICKSTART.md](#) troubleshooting section
- Review API documentation at `/docs`
- Inspect database with provided scripts
- Check GitHub issues for similar problems

Last Updated: April 23, 2026

Version: 1.2 - Barcode Scanner, Theme Support, Expanded Scripts & Reports